

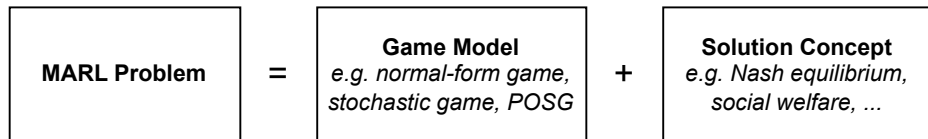
Multi-Agent Reinforcement Learning & Agentic AI

MARL in Games & Deep MARL Part I

Lecture Outline

- Learning framework for MARL
- Single-agent RL reduction
- Challenges of MARL
- Temporal-difference learning for games: joint-action learning
- Agent modelling

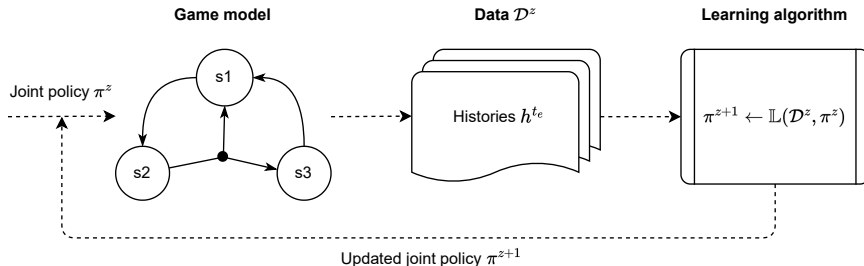
Recap: MARL Problem Definition



- Sequential decision process is defined formally as a **Game**
- When agents receive different rewards, maximizing returns of one agent is no longer enough to determine a solution
- We must consider the **joint policy** of all agents
- There are many different solution concepts

MARL Learning Framework

MARL Learning Process



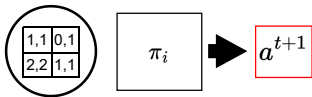
- The game model defines the learning environment
- Interaction data from joint policy π^z are collected as $\mathcal{D}^z = \{h^{te} \mid e = 1, \dots, z\}, z \geq 0$
- A learning algorithm updates the joint policy as $\pi^{z+1} = \mathbb{L}(\mathcal{D}^z, \pi^z)$
- The learning goal is a chosen solution concept, e.g. Nash equilibrium

MARL Learning Process - Continued

There are several additional nuances to the MARL learning process.

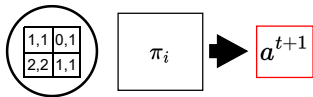
- The chosen game model will determine what the learnt joint policy conditions on:
 - Normal-form games: simple probability distribution across actions
 - Repeated normal-form games: condition on action histories $h^t = (a^0, \dots, a^{t-1})$
 - Stochastic games: condition on state-action histories $h^t = (s^0, a^0, s^1, a^1, \dots, s^t)$
 - POSG: condition on independent observation histories $h_i^t = (o_i^0, \dots, o_i^t)$
- Policies can however be adjusted as required e.g. in a stochastic game only condition on the last state-action
- The learning algorithm $\mathbb{L}$ can consists of multiple learning algorithms e.g. $\mathbb{L}_i \forall i \in I$
- Each $\mathbb{L}_i$ may use different parts of the data, or use independently collected data

Inputs of Policies Depend on Game Model

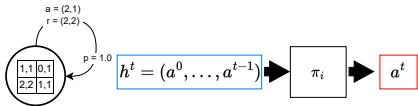


Normal-form games

Inputs of Policies Depend on Game Model

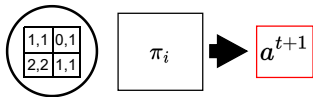


Normal-form games

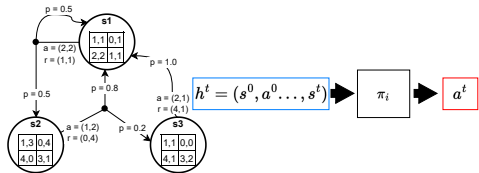


Repeated normal-form
games

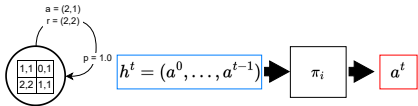
Inputs of Policies Depend on Game Model



Normal-form games

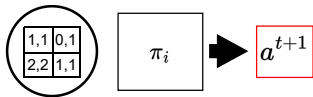


Stochastic games

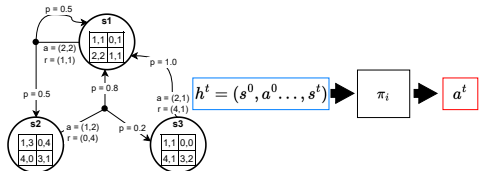


Repeated normal-form
games

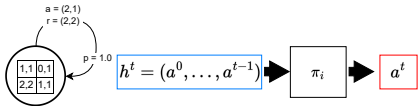
Inputs of Policies Depend on Game Model



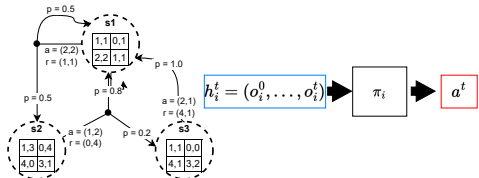
Normal-form games



Stochastic games



Repeated normal-form
games



Partially observable stochastic games

To evaluate the learning algorithm, we typically assess whether the learnt joint policy has **converged** to an optimal joint policy:

$$\lim_{z \rightarrow \infty} \pi^z = \pi^*$$

- Optimal joint policies may differ depending on the solution concept
- There may be many valid solutions $\Rightarrow$ e.g. multiple Nash equilibria
- In practice, we cannot collect infinite data $z \rightarrow \infty$
 $\Rightarrow$ Learning typically stops after a predefined 'budget' (e.g. training steps)

Single-Agent RL Reductions

The simplest way to apply RL algorithms in multi-agent settings is to reduce them to single-agent problems.

Single-Agent RL Reductions

The simplest way to apply RL algorithms in multi-agent settings is to reduce them to single-agent problems.

Central learning:

- Apply one single-agent RL algorithm to control all agents centrally
⇒ A central policy is learned over the joint action space

Single-Agent RL Reductions

The simplest way to apply RL algorithms in multi-agent settings is to reduce them to single-agent problems.

Central learning:

- Apply one single-agent RL algorithm to control all agents centrally
⇒ A central policy is learned over the joint action space

Independent learning:

- Apply single-agent RL algorithms to each agent independently
⇒ Agents do not explicitly consider or represent each other

Central Learning

Central learning: learn a single central policy π_c which receives observations of all agents and selects an action for each agent (i.e. joint action $(a_1, \dots, a_n)$).

- Requires transforming the joint reward $(r_1, \dots, r_n)$ into a single scalar reward r

Central learning: learn a single central policy π_c which receives observations of all agents and selects an action for each agent (i.e. joint action $(a_1, \dots, a_n)$).

- Requires transforming the joint reward $(r_1, \dots, r_n)$ into a single scalar reward r
- This can be easy in some settings, e.g. in games with common rewards $r = r_i$, but difficult in zero-sum or general-sum games

Central learning: learn a single central policy π_c which receives observations of all agents and selects an action for each agent (i.e. joint action $(a_1, \dots, a_n)$).

- Requires transforming the joint reward $(r_1, \dots, r_n)$ into a single scalar reward r
- This can be easy in some settings, e.g. in games with common rewards $r = r_i$, but difficult in zero-sum or general-sum games
- May not scale well with the number of agents, as the joint action space may grow exponentially with the number of agents

Central learning: learn a single central policy π_c which receives observations of all agents and selects an action for each agent (i.e. joint action $(a_1, \dots, a_n)$).

- Requires transforming the joint reward $(r_1, \dots, r_n)$ into a single scalar reward r
- This can be easy in some settings, e.g. in games with common rewards $r = r_i$, but difficult in zero-sum or general-sum games
- May not scale well with the number of agents, as the joint action space may grow exponentially with the number of agents
- May also not be suitable in environments that require agents to act independently based on local observations

Algorithm Central Q-learning

- 1: Initialize: $Q(s, a) = 0$ for all $s \in S$ and $a \in A = A_1 \times \dots \times A_n$
 - 2: Repeat for every episode:
 - 3: **for** $t = 0, 1, 2, \dots$ **do**
 - 4: Observe current state s^t
 - 5: With probability ϵ : choose random joint action $a^t \in A$
 - 6: Otherwise: choose joint action $a^t \in \arg \max_a Q(s^t, a)$
 - 7: Apply joint action a^t , observe rewards $r_1^t, \dots, r_n^t$ and next state s^{t+1}
 - 8: Transform $r_1^t, \dots, r_n^t$ into scalar reward r^t
 - 9: $Q(s^t, a^t) \leftarrow Q(s^t, a^t) + \alpha[r^t + \gamma \max_{a'} Q(s^{t+1}, a') - Q(s^t, a^t)]$
-

Independent Learning

Independent Learning

Independent learning: each agent i learns its policy π_i using only its local history of observations.

- From the perspective of each agent i , the environment transition function looks like this:

$$\mathcal{T}_i(s^{t+1}|s^t, a_i) \propto \sum_{a_{-i} \in A_{-i}} \mathcal{T}(s^{t+1}|s^t, \langle a_i, a_{-i} \rangle) \prod_{j \neq i} \pi_j(a_j|s^t)$$

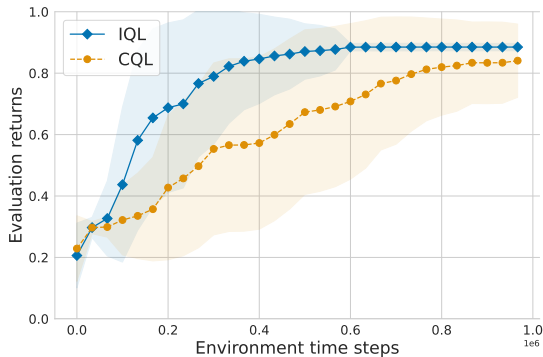
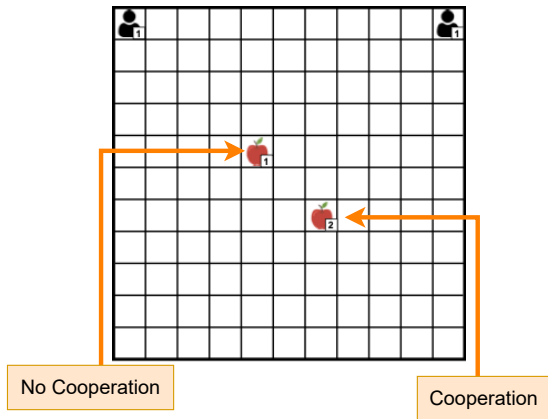
- As each agent j 's policies are updated, the action probabilities π_j change
 $\Rightarrow$ From agent i 's perspective, the transition function $\mathcal{T}_i$ appears non-stationary!

Independent Q-learning

Algorithm Independent Q-learning (IQL) for stochastic games

- 1: *// Algorithm controls agent i*
 - 2: Initialize: $Q_i(s, a_i) = 0$ for all $s \in S, a_i \in A_i$
 - 3: Repeat for every episode:
 - 4: **for** $t = 0, 1, 2, \dots$ **do**
 - 5: Observe current state s^t
 - 6: With probability ϵ : choose random action $a_i^t \in A_i$
 - 7: Otherwise: choose action $a_i^t \in \arg \max_{a_i} Q_i(s^t, a_i)$
 - 8: (meanwhile, other agents $j \neq i$ choose their actions a_j^t)
 - 9: Observe own reward r_i^t and next state s^{t+1}
 - 10: $Q_i(s^t, a_i^t) \leftarrow Q_i(s^t, a_i^t) + \alpha[r_i^t + \gamma \max_{a'_i} Q_i(s^{t+1}, a'_i) - Q_i(s^t, a_i^t)]$
-

IQL and CQL in Level-Based Foraging



- IQL can learn more quickly, as CQL needs to explore $6^2 = 36$ actions in each state

Independent Deep Q-Networks

Algorithm Independent deep Q-networks

- 1: Initialize n value networks with random parameters $\theta_1, \dots, \theta_n$
 - 2: Initialize n target networks with parameters $\bar{\theta}_1 = \theta_1, \dots, \bar{\theta}_n = \theta_n$
 - 3: Initialize a replay buffer for each agent $D_1, D_2, \dots, D_n$
 - 4: **for** time step $t = 0, 1, 2, \dots$ **do**
 - 5: Collect current observations $o_1^t, \dots, o_n^t$
 - 6: **for** agent $i = 1, \dots, n$ **do**
 - 7: With probability ϵ : choose random action a_i^t
 - 8: Otherwise: choose $a_i^t \in \arg \max_{a_i} Q(h_i^t, a_i; \theta_i)$
 - 9: Apply actions $(a_1^t, \dots, a_n^t)$; collect rewards $r_1^t, \dots, r_n^t$ and next observations $o_1^{t+1}, \dots, o_n^{t+1}$
 - 10: **for** agent $i = 1, \dots, n$ **do**
 - 11: Store transition $(h_i^t, a_i^t, r_i^t, h_i^{t+1})$ in replay buffers D_i
 - 12: Sample random mini-batch of B transitions $(h_i^k, a_i^k, r_i^k, h_i^{k+1})$ from D_i
 - 13: **if** s^{k+1} is terminal **then**
 - 14: Targets $y_i^k \leftarrow r_i^k$
 - 15: **else**
 - 16: Targets $y_i^k \leftarrow r_i^k + \gamma \max_{a'_i \in A_i} Q(h_i^{k+1}, a'_i; \bar{\theta}_i)$
 - 17: Loss $\mathcal{L}(\theta_i) \leftarrow \frac{1}{B} \sum_{k=1}^B \left(y_i^k - Q(h_i^k, a_i^k; \theta_i) \right)^2$
 - 18: Update parameters θ_i by minimizing the loss $\mathcal{L}(\theta_i)$
 - 19: In a set interval, update target network parameters $\bar{\theta}_i$
-

Independent Deep Q-Networks

Algorithm Independent deep Q-networks

- 1: Initialize n value networks with random parameters $\theta_1, \dots, \theta_n$
 - 2: Initialize n target networks with parameters $\bar{\theta}_1 = \theta_1, \dots, \bar{\theta}_n = \theta_n$
 - 3: Initialize a replay buffer for each agent $D_1, D_2, \dots, D_n$
 - 4: **for** time step $t = 0, 1, 2, \dots$ **do**
 - 5: Collect current observations $o_1^t, \dots, o_n^t$
 - 6: **for** agent $i = 1, \dots, n$ **do**
 - 7: With probability ϵ : choose random action a_i^t
 - 8: Otherwise: choose $a_i^t \in \arg \max_{a_i} Q(h_i^t, a_i; \theta_i)$
 - 9: Apply actions $(a_1^t, \dots, a_n^t)$; collect rewards $r_1^t, \dots, r_n^t$ and next observations $o_1^{t+1}, \dots, o_n^{t+1}$
 - 10: **for** agent $i = 1, \dots, n$ **do**
 - 11: Store transition $(h_i^t, a_i^t, r_i^t, h_i^{t+1})$ in replay buffers D_i
 - 12: Sample random mini-batch of B transitions $(h_i^k, a_i^k, r_i^k, h_i^{k+1})$ from D_i
 - 13: **if** s^{k+1} is terminal **then**
 - 14: Targets $y_i^k \leftarrow r_i^k$
 - 15: **else**
 - 16: Targets $y_i^k \leftarrow r_i^k + \gamma \max_{a'_i \in A_i} Q(h_i^{k+1}, a'_i; \bar{\theta}_i)$
 - 17: Loss $\mathcal{L}(\theta_i) \leftarrow \frac{1}{B} \sum_{k=1}^B \left(y_i^k - Q(h_i^k, a_i^k; \theta_i) \right)^2$
 - 18: Update parameters θ_i by minimizing the loss $\mathcal{L}(\theta_i)$
 - 19: In a set interval, update target network parameters $\bar{\theta}_i$
-

Independent Deep Q-Networks

Algorithm Independent deep Q-networks

```
1: Initialize  $n$  value networks with random parameters  $\theta_1, \dots, \theta_n$ 
2: Initialize  $n$  target networks with parameters  $\bar{\theta}_1 = \theta_1, \dots, \bar{\theta}_n = \theta_n$ 
3: Initialize a replay buffer for each agent  $D_1, D_2, \dots, D_n$ 
4: for time step  $t = 0, 1, 2, \dots$  do
5:   Collect current observations  $o_1^t, \dots, o_n^t$ 
6:   for agent  $i = 1, \dots, n$  do
7:     With probability  $\epsilon$ : choose random action  $a_i^t$ 
8:     Otherwise: choose  $a_i^t \in \arg \max_{a_i} Q(h_i^t, a_i; \theta_i)$ 
9:   Apply actions  $(a_1^t, \dots, a_n^t)$ ; collect rewards  $r_1^t, \dots, r_n^t$  and next observations  $o_1^{t+1}, \dots, o_n^{t+1}$ 
10:  for agent  $i = 1, \dots, n$  do
11:    Store transition  $(h_i^t, a_i^t, r_i^t, h_i^{t+1})$  in replay buffers  $D_i$ 
12:    Sample random mini-batch of  $B$  transitions  $(h_i^k, a_i^k, r_i^k, h_i^{k+1})$  from  $D_i$ 
13:    if  $s^{k+1}$  is terminal then
14:      Targets  $y_i^k \leftarrow r_i^k$ 
15:    else
16:      Targets  $y_i^k \leftarrow r_i^k + \gamma \max_{a'_i \in A_i} Q(h_i^{k+1}, a'_i; \bar{\theta}_i)$ 
17:      Loss  $\mathcal{L}(\theta_i) \leftarrow \frac{1}{B} \sum_{k=1}^B \left( y_i^k - Q(h_i^k, a_i^k; \theta_i) \right)^2$ 
18:      Update parameters  $\theta_i$  by minimizing the loss  $\mathcal{L}(\theta_i)$ 
19:      In a set interval, update target network parameters  $\bar{\theta}_i$ 
```

- Replay buffer contains off-policy experiences due to changing policies
- In MARL, the policies of **all** agents are changing $\rightarrow$ training can be unstable

Independent Advantage Actor-Critic

Algorithm Independent A2C with synchronous environments

- 1: Initialize n actor networks with random parameters $\phi_1, \dots, \phi_n$
 - 2: Initialize n critic networks with random parameters $\theta_1, \dots, \theta_n$
 - 3: Initialize K parallel environments
 - 4: **for** time step $t = 0 \dots$ **do**
 - 5: Batch of observations for each agent and environment:
$$\begin{bmatrix} o_1^{t,1} & \dots & o_1^{t,K} \\ & \ddots & \\ o_n^{t,1} & \dots & o_n^{t,K} \end{bmatrix}$$
 - 6: Sample actions $\begin{bmatrix} a_1^{t,1} & \dots & a_1^{t,K} \\ & \ddots & \\ a_n^{t,1} & \dots & a_n^{t,K} \end{bmatrix} \sim \pi(\cdot \mid h_1^t; \phi_1), \dots, \pi(\cdot \mid h_n^t; \phi_n)$
 - 7: Apply actions; collect rewards $\begin{bmatrix} r_1^{t,1} & \dots & r_1^{t,K} \\ & \ddots & \\ r_n^{t,1} & \dots & r_n^{t,K} \end{bmatrix}$ and observations $\begin{bmatrix} o_1^{t+1,1} & \dots & o_1^{t+1,K} \\ & \ddots & \\ o_n^{t+1,1} & \dots & o_n^{t+1,K} \end{bmatrix}$
 - 8: **for** agent $i = 1, \dots, n$ **do**
 - 9: **if** $s^{t+1,k}$ is terminal **then**
 - 10: Advantage $Adv(h_i^{t,k}, a_i^{t,k}) \leftarrow r_i^{t,k} - V(h_i^{t,k}; \theta_i)$
 - 11: Critic target $y_i^{t,k} \leftarrow r_i^{t,k}$
 - 12: **else**
 - 13: Advantage $Adv(h_i^{t,k}, a_i^{t,k}) \leftarrow r_i^{t,k} + \gamma V(h_i^{t+1,k}; \theta_i) - V(h_i^{t,k}; \theta_i)$
 - 14: Critic target $y_i^{t,k} \leftarrow r_i^{t,k} + \gamma V(h_i^{t+1,k}; \theta_i)$
 - 15: Actor loss $\mathcal{L}(\phi_i) \leftarrow \frac{1}{K} \sum_{k=1}^K Adv(h_i^{t,k}, a_i^{t,k}) \log \pi(a_i^{t,k} \mid h_i^{t,k}; \phi_i)$
 - 16: Critic loss $\mathcal{L}(\theta_i) \leftarrow \frac{1}{K} \sum_{k=1}^K (y_i^{t,k} - V(h_i^{t,k}; \theta_i))^2$
 - 17: Update parameters ϕ_i by minimizing the actor loss $\mathcal{L}(\phi_i)$
 - 18: Update parameters θ_i by minimizing the critic loss $\mathcal{L}(\theta_i)$
-

Independent Advantage Actor-Critic

Algorithm Independent A2C with synchronous environments

- 1: Initialize n actor networks with random parameters $\phi_1, \dots, \phi_n$
 - 2: Initialize n critic networks with random parameters $\theta_1, \dots, \theta_n$
 - 3: Initialize K parallel environments
 - 4: **for** time step $t = 0 \dots$ **do**
 - 5: Batch of observations for each agent and environment: $\begin{bmatrix} o_1^{t,1} \dots o_1^{t,K} \\ \vdots \\ o_n^{t,1} \dots o_n^{t,K} \end{bmatrix}$
 - 6: Sample actions $\begin{bmatrix} a_1^{t,1} \dots a_1^{t,K} \\ \vdots \\ a_n^{t,1} \dots a_n^{t,K} \end{bmatrix} \sim \pi(\cdot \mid h_1^t; \phi_1), \dots, \pi(\cdot \mid h_n^t; \phi_n)$
 - 7: Apply actions; collect rewards $\begin{bmatrix} r_1^{t,1} \dots r_1^{t,K} \\ \vdots \\ r_n^{t,1} \dots r_n^{t,K} \end{bmatrix}$ and observations $\begin{bmatrix} o_1^{t+1,1} \dots o_1^{t+1,K} \\ \vdots \\ o_n^{t+1,1} \dots o_n^{t+1,K} \end{bmatrix}$
 - 8: **for** agent $i = 1, \dots, n$ **do**
 - 9: **if** $s^{t+1,k}$ is terminal **then**
 - 10: Advantage $Adv(h_i^{t,k}, a_i^{t,k}) \leftarrow r_i^{t,k} - V(h_i^{t,k}; \theta_i)$
 - 11: Critic target $y_i^{t,k} \leftarrow r_i^{t,k}$
 - 12: **else**
 - 13: Advantage $Adv(h_i^{t,k}, a_i^{t,k}) \leftarrow r_i^{t,k} + \gamma V(h_i^{t+1,k}; \theta_i) - V(h_i^{t,k}; \theta_i)$
 - 14: Critic target $y_i^{t,k} \leftarrow r_i^{t,k} + \gamma V(h_i^{t+1,k}; \theta_i)$
 - 15: Actor loss $\mathcal{L}(\phi_i) \leftarrow \frac{1}{K} \sum_{k=1}^K Adv(h_i^{t,k}, a_i^{t,k}) \log \pi(a_i^{t,k} \mid h_i^{t,k}; \phi_i)$
 - 16: Critic loss $\mathcal{L}(\theta_i) \leftarrow \frac{1}{K} \sum_{k=1}^K (y_i^{t,k} - V(h_i^{t,k}; \theta_i))^2$
 - 17: Update parameters ϕ_i by minimizing the actor loss $\mathcal{L}(\phi_i)$
 - 18: Update parameters θ_i by minimizing the critic loss $\mathcal{L}(\theta_i)$
-

- Almost identical to single-agent A2C

Independent Advantage Actor-Critic

Algorithm Independent A2C with synchronous environments

```

1: Initialize  $n$  actor networks with random parameters  $\phi_1, \dots, \phi_n$ 
2: Initialize  $n$  critic networks with random parameters  $\theta_1, \dots, \theta_n$ 
3: Initialize  $K$  parallel environments
4: for time step  $t = 0 \dots$  do

5:   Batch of observations for each agent and environment:
       
$$\begin{bmatrix} o_1^{t,1} & \dots & o_1^{t,K} \\ & \ddots & \\ o_n^{t,1} & \dots & o_n^{t,K} \end{bmatrix}$$


6:   Sample actions  $\begin{bmatrix} a_1^{t,1} & \dots & a_1^{t,K} \\ & \ddots & \\ a_n^{t,1} & \dots & a_n^{t,K} \end{bmatrix} \sim \pi(\cdot \mid h_1^t; \phi_1), \dots, \pi(\cdot \mid h_n^t; \phi_n)$ 

7:   Apply actions; collect rewards  $\begin{bmatrix} r_1^{t,1} & \dots & r_1^{t,K} \\ & \ddots & \\ r_n^{t,1} & \dots & r_n^{t,K} \end{bmatrix}$  and observations  $\begin{bmatrix} o_1^{t+1,1} & \dots & o_1^{t+1,K} \\ & \ddots & \\ o_n^{t+1,1} & \dots & o_n^{t+1,K} \end{bmatrix}$ 

8:   for agent  $i = 1, \dots, n$  do
9:     if  $s^{t+1,k}$  is terminal then
10:       Advantage  $Adv(h_i^{t,k}, a_i^{t,k}) \leftarrow r_i^{t,k} - V(h_i^{t,k}; \theta_i)$ 
11:       Critic target  $y_i^{t,k} \leftarrow r_i^{t,k}$ 
12:     else
13:       Advantage  $Adv(h_i^{t,k}, a_i^{t,k}) \leftarrow r_i^{t,k} + \gamma V(h_i^{t+1,k}; \theta_i) - V(h_i^{t,k}; \theta_i)$ 
14:       Critic target  $y_i^{t,k} \leftarrow r_i^{t,k} + \gamma V(h_i^{t+1,k}; \theta_i)$ 
15:       Actor loss  $\mathcal{L}(\phi_i) \leftarrow \frac{1}{K} \sum_{k=1}^K Adv(h_i^{t,k}, a_i^{t,k}) \log \pi(a_i^{t,k} \mid h_i^{t,k}; \phi_i)$ 
16:       Critic loss  $\mathcal{L}(\theta_i) \leftarrow \frac{1}{K} \sum_{k=1}^K (y_i^{t,k} - V(h_i^{t,k}; \theta_i))^2$ 
17:       Update parameters  $\phi_i$  by minimizing the actor loss  $\mathcal{L}(\phi_i)$ 
18:       Update parameters  $\theta_i$  by minimizing the critic loss  $\mathcal{L}(\theta_i)$ 

```

- Almost identical to single-agent A2C
- Similar adaptation can be done for independent REINFORCE and independent PPO

Modes of Operation in MARL

Modes of operation in MARL:

Self-play:

- *Algorithm self-play*: all agents use the same learning algorithm (and parameters)
- *Policy self-play*: agent's policy is trained directly against itself

Mixed-play:

- Agents use different learning algorithms
- Trading bots in markets that use RL are one example of a mixed-play scenario

MARL Challenges

Singe-Agent RL Challenges

- Unknown environment dynamics
- Exploration-exploitation dilemma
- Non-stationarity from bootstrapping
- Temporal credit assignment

MARL Challenges

Singe-Agent RL Challenges

- Unknown environment dynamics
- Exploration-exploitation dilemma
- Non-stationarity from bootstrapping
- Temporal credit assignment

Multi-Agent RL Challenges

- Non-stationarity from multiple learning agents
- Equilibrium selection
- Multi-agent credit assignment
- Scaling to many agents

Non-Stationarity

A stochastic process $X^t_{t \in \mathbb{N}^0}$ is stationary if:

- The probability distribution of $X^{t+\tau}$ does not depend on $\tau \in \mathbb{N}^0$, where t and $t + \tau$ are time indices
- This means that the dynamics of the process do not change over time

Non-Stationarity

A stochastic process $X^t_{t \in \mathbb{N}^0}$ is stationary if:

- The probability distribution of $X^{t+\tau}$ does not depend on $\tau \in \mathbb{N}^0$, where t and $t + \tau$ are time indices
- This means that the dynamics of the process do not change over time

Consider: X^t samples the state s^t at each time step t :

- In a MDP, X^t is completely defined by the state transition function $\mathcal{T}(s^t | s^{t-1}, a^{t-1})$ and the agent's policy π which selects an action $a \sim \pi(.|s)$

Non-Stationarity

A stochastic process $X^t_{t \in \mathbb{N}^0}$ is stationary if:

- The probability distribution of $X^{t+\tau}$ does not depend on $\tau \in \mathbb{N}^0$, where t and $t + \tau$ are time indices
- This means that the dynamics of the process do not change over time

Consider: X^t samples the state s^t at each time step t :

- In a MDP, X^t is completely defined by the state transition function $\mathcal{T}(s^t | s^{t-1}, a^{t-1})$ and the agent's policy π which selects an action $a \sim \pi(.|s)$
- If π does not change, then this process is stationary (i.e. independent of t) because s^t depends only on s^{t-1} , a^{t-1} and a^{t-1} depends only on s^{t-1} via $\pi(.|s^{t-1})$

Non-Stationarity

A stochastic process $X^t_{t \in \mathbb{N}^0}$ is stationary if:

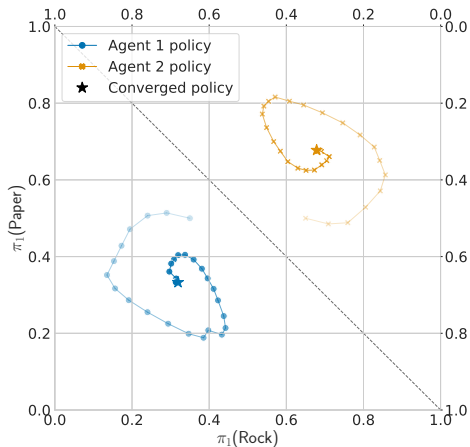
- The probability distribution of $X^{t+\tau}$ does not depend on $\tau \in \mathbb{N}^0$, where t and $t + \tau$ are time indices
- This means that the dynamics of the process do not change over time

Consider: X^t samples the state s^t at each time step t :

- In a MDP, X^t is completely defined by the state transition function $\mathcal{T}(s^t | s^{t-1}, a^{t-1})$ and the agent's policy π which selects an action $a \sim \pi(.|s)$
- If π does not change, then this process is stationary (i.e. independent of t) because s^t depends only on s^{t-1} , a^{t-1} and a^{t-1} depends only on s^{t-1} via $\pi(.|s^{t-1})$
- However, in RL the policy does change over time through the learning $\pi^{z+1} = \mathbb{L}(\mathcal{D}^z, \pi^z)$, which leads to **non-stationarity** in X^t

Non-Stationarity in Multi-Agent Settings

In MARL, non-stationarity is exacerbated by multiple agents changing their policies!



- $\pi^{z+1} = \mathbb{L}(\mathcal{D}^z, \pi^z)$ updates an *entire* joint policy $\pi^z = (\pi_1^z, \dots, \pi_n^z)$
- Environment is non-stationary from each agent's perspective
- Can cause cyclic learning dynamics where agents co-adapt to each other's changing policies

Equilibrium Selection

Equilibrium selection: a game may have multiple equilibria, which can yield different expected returns to the agents.

- **Example:** Stag Hunt matrix game
- Two hunters choose: cooperate to hunt stag (S) or go solo for hare (H)

	S	H
S	4,4	0,3
H	3,0	2,2

Equilibrium Selection

Equilibrium selection: a game may have multiple equilibria, which can yield different expected returns to the agents.

- **Example:** Stag Hunt matrix game
- Two hunters choose: cooperate to hunt stag (S) or go solo for hare (H)
- Nash equilibria: reward-dominant (S,S)
maximizes reward, risk-dominant (H,H)
minimizes risk

	S	H
S	4,4	0,3
H	3,0	2,2

Equilibrium Selection

Equilibrium selection: a game may have multiple equilibria, which can yield different expected returns to the agents.

- **Example:** Stag Hunt matrix game
- Two hunters choose: cooperate to hunt stag (S) or go solo for hare (H)
- Nash equilibria: reward-dominant (S,S) maximizes reward, risk-dominant (H,H) minimizes risk
- (S,S) requires trust; (H,H) offers a safe, lower reward

	S	H
S	4,4	0,3
H	3,0	2,2

Equilibrium Selection

Equilibrium selection: a game may have multiple equilibria, which can yield different expected returns to the agents.

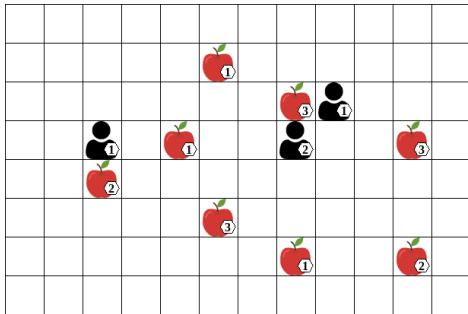
- **Example:** Stag Hunt matrix game
- Two hunters choose: cooperate to hunt stag (S) or go solo for hare (H)
- Nash equilibria: reward-dominant (S,S) maximizes reward, risk-dominant (H,H) minimizes risk
- (S,S) requires trust; (H,H) offers a safe, lower reward

	S	H
S	4,4	0,3
H	3,0	2,2

- Indep. Q-learning may bias towards (H,H) due to initial action uncertainty
- Early random actions can reinforce (H,H) since deviating from H is penalized if the other agent chooses H

Multi-Agent Credit Assignment

Multi-agent credit assignment: which agent's actions contributed to received rewards?



- At time step t all agents perform *collect*, each receiving reward $+1$
- Whose actions led to the reward?
- The agent on the left did not contribute
- For a learning agent that only observes s^t, a^t, r^t, s^{t+1} , disentangling the action contributions is difficult

Joint Actions for Addressing Multi-Agent Credit Assignment

Joint actions can help disentangle agent contributions. Consider the RPS game:

	R	P	S
R	0,0	-1,1	1,-1
P	1,-1	0,0	-1,1
S	-1,1	1,-1	0,0

Joint Actions for Addressing Multi-Agent Credit Assignment

Joint actions can help disentangle agent contributions. Consider the RPS game:

	R	P	S
R	0,0	-1,1	1,-1
P	1,-1	0,0	-1,1
S	-1,1	1,-1	0,0

1. Agents choose actions $(a_1, a_2) = (R, S) \Rightarrow$ agent 1 receives reward +1

Joint Actions for Addressing Multi-Agent Credit Assignment

Joint actions can help disentangle agent contributions. Consider the RPS game:

	R	P	S
R	0,0	-1,1	1,-1
P	1,-1	0,0	-1,1
S	-1,1	1,-1	0,0

1. Agents choose actions $(a_1, a_2) = (R, S) \Rightarrow$ agent 1 receives reward $+1$
2. Then agents choose $(a_1, a_2) = (R, P) \Rightarrow$ agent 1 receives reward -1

Joint Actions for Addressing Multi-Agent Credit Assignment

Joint actions can help disentangle agent contributions. Consider the RPS game:

	R	P	S
R	0,0	-1,1	1,-1
P	1,-1	0,0	-1,1
S	-1,1	1,-1	0,0

1. Agents choose actions $(a_1, a_2) = (R, S) \Rightarrow$ agent 1 receives reward $+1$
2. Then agents choose $(a_1, a_2) = (R, P) \Rightarrow$ agent 1 receives reward -1
3. Action value $Q(a_1)$ does not model agent 2, value for action R appears to be 0

Joint Actions for Addressing Multi-Agent Credit Assignment

Joint actions can help disentangle agent contributions. Consider the RPS game:

	R	P	S
R	0,0	-1,1	1,-1
P	1,-1	0,0	-1,1
S	-1,1	1,-1	0,0

1. Agents choose actions $(a_1, a_2) = (R, S) \Rightarrow$ agent 1 receives reward $+1$
2. Then agents choose $(a_1, a_2) = (R, P) \Rightarrow$ agent 1 receives reward -1
3. Action value $Q(a_1)$ does not model agent 2, value for action R appears to be 0
4. **Joint action value model** $Q_1(a_1, a_2)$ can represent the effect of agent 2:
 $Q_1(R, S) = +1$ and $Q_1(R, P) = -1$

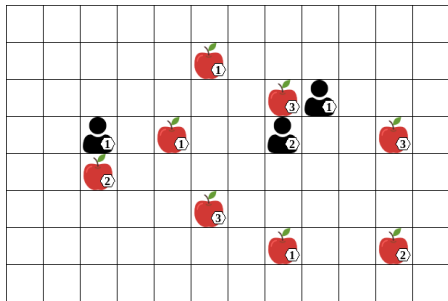
Problem

- **Joint action space** can grow **exponentially** with number of agents:

$$|A| = |A_1| \cdot \dots \cdot |A_n|$$

- If agents have associated features in s (e.g. agent position) then $|S|$ also increases exponentially
⇒ How to scale efficiently to many agents?

Scaling to Many Agents



Changing number of agents from 3 to 5 increases the number of joint actions from 216 to 7776!

Temporal-difference learning for games: joint-action learning

From Dynamic Programming to Temporal-Difference Learning

Shapley (1953) proposed **value iteration** via dynamic programming to compute **minimax** joint policy in zero-sum stochastic games with two agents.

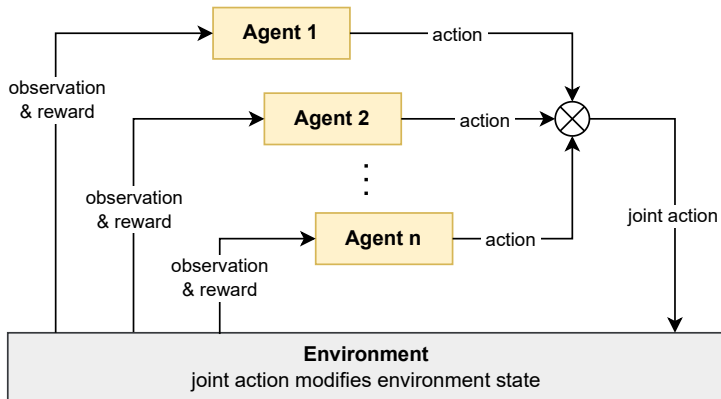
Problem

Dynamic programming requires full knowledge of game

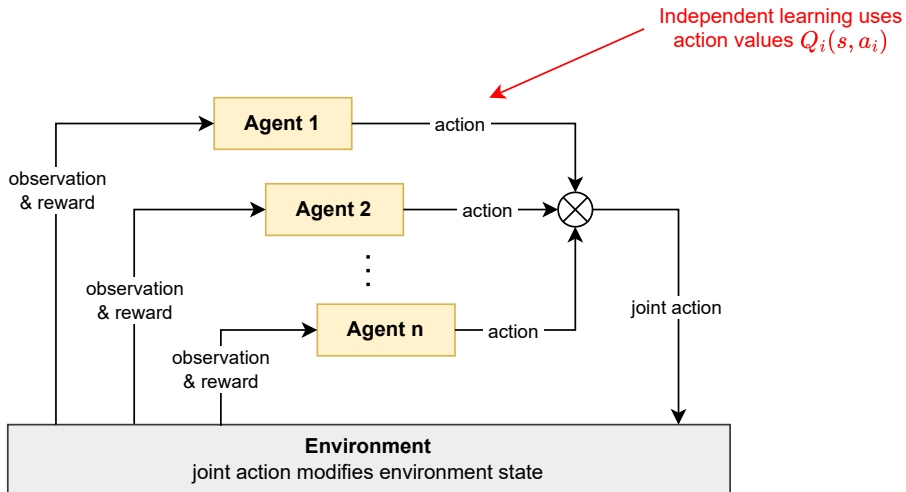
- Including reward functions $\mathcal{R}_i$ and state transition function $\mathcal{T}$
- May not be available!

Can we **learn** equilibrium joint policy via **temporal-difference learning**?

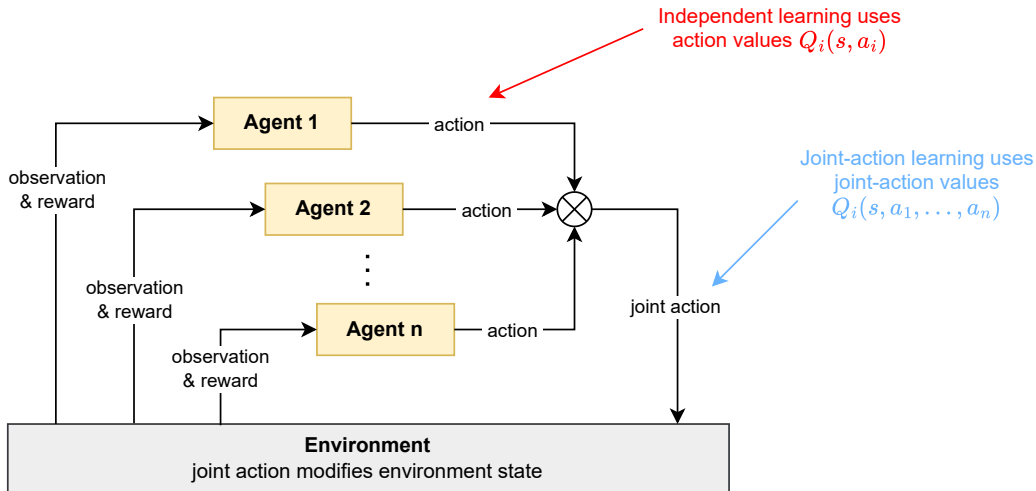
Temporal-Difference Learning for Games: Joint-Action Learning



Temporal-Difference Learning for Games: Joint-Action Learning



Temporal-Difference Learning for Games: Joint-Action Learning



Temporal-Difference Learning for Games: Joint-Action Learning

Problem

$Q_i(s, a_1, \dots, a_n)$ is not enough to find optimal action for agent i

- Cannot evaluate $\max_{a_i} Q_i(s, a_1, \dots, a_n)$

$\Rightarrow$ Optimal action depends on actions of other agents!

Problem

$Q_i(s, a_1, \dots, a_n)$ is not enough to find optimal action for agent i

- Cannot evaluate $\max_{a_i} Q_i(s, a_1, \dots, a_n)$

$\Rightarrow$ Optimal action depends on actions of other agents!

We have to define:

1. How to select action from Q_i ?
2. How to update Q_i ?

Agent Modeling & Best Response

Game theory solutions are **normative**: they *prescribe* how agents *should* behave

- e.g. minimax assumes worst-case opponent

Agent Modeling & Best Response

Game theory solutions are **normative**: they *prescribe* how agents *should* behave

- e.g. minimax assumes worst-case opponent

What if agents don't behave as prescribed by solution?

Agent Modeling & Best Response

Game theory solutions are **normative**: they *prescribe* how agents *should* behave

- e.g. minimax assumes worst-case opponent

What if agents don't behave as prescribed by solution? Other approach: **agent modeling with best response**

- Learn models of other agents to predict their actions
- Compute optimal action (best response) against agent models



Many kinds of agent modeling:

- Policy reconstruction
- Type-based reasoning
- Classification
- Plan recognition
- Recursive reasoning
- Graphical methods
- Group modeling
- Implicit modeling

S. Albrecht, P. Stone. **Autonomous Agents Modelling Other Agents: A Comprehensive Survey and Open Problems.** *Artificial Intelligence*, 2018

Policy Reconstruction & Best Response

Policy reconstruction: learn model $\hat{\pi}_j \approx \pi_j$ from past observed actions

In general, can train model with supervised learning on data $\{(s^\tau, a_j^\tau)\}_{\tau=1}^{t-1}$

- E.g. look-up table, neural network, finite state machine, ...
- Model should support incremental updating

Policy Reconstruction & Best Response

Policy reconstruction: learn model $\hat{\pi}_j \approx \pi_j$ from past observed actions

In general, can train model with **supervised learning** on data $\{(s^\tau, a_j^\tau)\}_{\tau=1}^{t-1}$

- E.g. look-up table, neural network, finite state machine, ...
- Model should support incremental updating

Given models for other agents $\hat{\pi}_{-i} = \{\hat{\pi}_j\}_{j \neq i}$, compute **best response**

$$\pi_i \in \text{BR}_i(\hat{\pi}_{-i})$$

Joint-Action Learning with Agent Modeling

Each agent i models other agents j as stationary distribution in a **stochastic game** by using **joint-action learning with agent models**

Joint-Action Learning with Agent Modeling

Each agent i models other agents j as stationary distribution in a **stochastic game** by using **joint-action learning with agent models**

Agents models other agents j , this time conditioned on states s :

$$\hat{\pi}_j(a_j | s) = \frac{C(s, a_j)}{\sum_{a'_j \in A_j} C(s, a'_j)}$$

Joint-Action Learning with Agent Modeling

Each agent i models other agents j as stationary distribution in a **stochastic game** by using **joint-action learning with agent models**

Agents models other agents j , this time conditioned on states s :

$$\hat{\pi}_j(a_j | s) = \frac{C(s, a_j)}{\sum_{a'_j \in A_j} C(s, a'_j)}$$

Given models $\{\hat{\pi}_j\}_{j \neq i}$, action values are defined as:

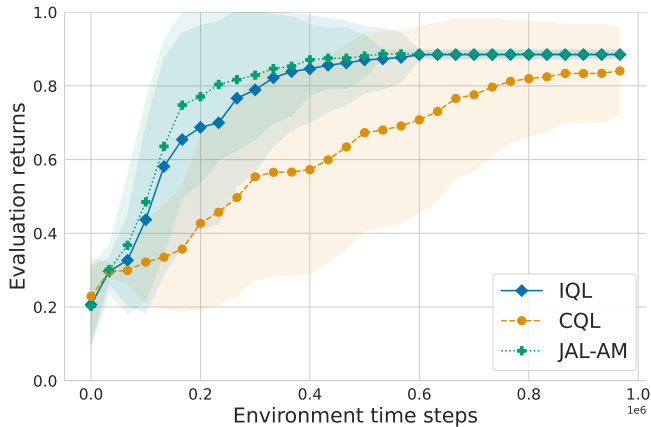
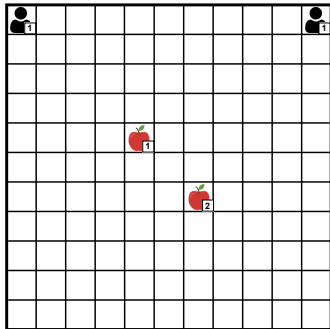
$$AV_i(s, a_i) = \sum_{a_{-i} \in A_{-i}} Q_i(s, \langle a_i, a_{-i} \rangle) \prod_{j \neq i} \hat{\pi}_j(a_j | s)$$

$\Rightarrow$ Use AV_i to select optimal actions and as learning update targets

Algorithm Joint-action learning with agent modeling (JAL-AM)

- 1: Initialize:
 - 2: $Q_i(s, a) = 0$ for all $s \in S, a \in A$
 - 3: Agent models $\hat{\pi}_j(a_j | s) = \frac{1}{|A_j|}$ for all $j \neq i, a_j \in A_j, s \in S$
 - 4: Repeat for every episode:
 - 5: **for** $t = 0, 1, 2, \dots$ **do**
 - 6: Observe current state s^t
 - 7: With probability ϵ : choose random action a_i^t
 - 8: Otherwise: choose best-response action $a_i^t \in \arg \max_{a_i} AV_i(s^t, a_i)$
 - 9: Observe joint action $a^t = (a_1^t, \dots, a_n^t)$, reward r_i^t , next state s^{t+1}
 - 10: Update agent models $\hat{\pi}_j$ with new observations (e.g., (s^t, a_j^t))
 - 11: $Q_i(s^t, a^t) \leftarrow Q_i(s^t, a^t) + \alpha \left[r_i^t + \gamma \max_{a'_i} AV_i(s^{t+1}, a'_i) - Q_i(s^t, a^t) \right]$
-

JAL-AM in Level-Based Foraging



Policy-Based Learning

All algorithms so far derive policies from learned **action-value functions**

- Has important limitations, e.g. algorithms using best-response actions from values (e.g. fictitious play, JAL-AM) cannot represent probabilistic equilibria
- Unable to represent uniform-random NE in Rock-Paper-Scissors

Policy-Based Learning

All algorithms so far derive policies from learned **action-value functions**

- Has important limitations, e.g. algorithms using best-response actions from values (e.g. fictitious play, JAL-AM) cannot represent probabilistic equilibria
- Unable to represent uniform-random NE in Rock-Paper-Scissors

Policy-based learning instead uses learning data to directly optimise policies

- Use *parameterised policies* that are differentiable
- Use *gradient-ascent techniques* to optimise parameters
 - ⇒ Can directly learn action probabilities in policies! (not covered in lectures)

We covered:

- Learning framework for MARL
- Single-agent RL reduction
- Challenges of MARL
- Temporal-difference learning for games: joint-action learning
- Centralised critics

Next we'll cover:

- Deep MARL - Part II